

Especificación de Requisitos y Plan de Implementación: Agente Consultor de IA

INDICE

1. Resumen del Objetivo del Agente Consultor

El propósito de este documento es definir el alcance, los requisitos funcionales y el plan de implementación inicial para el Agente Consultor de IA, sirviendo como la única fuente de verdad para el equipo de desarrollo. Su propósito es definir con precisión el alcance del proyecto, alinear al equipo de desarrollo en torno a un conjunto claro de requisitos y proporcionar una hoja de ruta accionable para la implementación, siguiendo una metodología Agile.

La visión del producto es crear una aplicación "agentica" diseñada para interactuar con usuarios empresariales. El sistema comprenderá el contexto de un problema de negocio mediante una serie de formularios dinámicos y, una vez recopilada la información suficiente, guiará la generación de una propuesta de solución utilizando Inteligencia Artificial. La aplicación se fundamenta en la gestión de sesiones de usuario, heurísticas de completitud para determinar cuándo se tiene suficiente información y memoria conversacional para asegurar una experiencia fluida y coherente.

Para estructurar este desarrollo de manera organizada, el proyecto se ha desglosado en las siguientes Épicas, que agrupan las funcionalidades clave y guían la planificación de los ciclos de desarrollo.

2. Épicas del Proyecto

En la gestión de proyectos Agile, las Épicas son grandes bloques de trabajo que agrupan Historias de Usuario relacionadas en torno a una funcionalidad principal. Sirven como contenedores de alto nivel para organizar el backlog del producto, facilitar la planificación estratégica a largo plazo y comunicar los objetivos de negocio al equipo de desarrollo.

A continuación, se presenta la tabla de Épicas que estructuran el alcance de este proyecto:

Código de épica	Épica
1	Infraestructura y autenticación
2	Motor de formularios dinámicos
3	Agente de IA empresarial
4	Gestión y sesión de estado
5	Generación de propuesta

3. Historias de Usuario y Criterios de Aceptación

Este apartado detalla el backlog de producto, traduciendo la visión del proyecto en requisitos funcionales concretos desde la perspectiva del usuario. Cada Historia de Usuario define un requisito funcional, mientras que los Criterios de Aceptación establecen las condiciones exactas que el software debe cumplir para que dicha funcionalidad se considere completa y validada.

Ident.	Descripción	Cód. Épica
HU.1	Como usuario, quiero poder describir el problema que tiene mi negocio.	2
HU.2	Quiero que la aplicación genere preguntas específicas basadas en respuestas anteriores.	3
HU.3	Quiero que el sistema guarde automáticamente el progreso de respuesta para retomar la consulta más tarde.	4
HU.4	Quiero que el sistema detecte cuando ya se tiene toda la información necesaria.	3
HU.5	Quiero recibir una propuesta de solución generada por la IA basado en toda la información que he suministrado	3
HU.6	Como nuevo usuario, quiero crear una cuenta introduciendo mis datos básicos, para poder tener una identidad en la plataforma.	1
HU.7	Como usuario registrado, quiero autenticarme con mis credenciales, para acceder a las funcionalidades privadas del agente consultor.	1
HU.8	Quiero poder acceder a los informes generados anteriormente.	4
HU.9	Quiero tener un razonamiento de por qué ha propuesto las soluciones en base al problema explicado.	3
HU.10	Quiero que la aplicación recuerde mis respuestas anteriores para que no me vuelva a formular preguntas que ya he contestado previamente.	3
HU.11	Quiero poder recibir un informe con un resumen de la sesión y propuesta de solución.	5
HU.12	Como usuario quiero poder descargarme el informe que ha generado la aplicación o enviarla vía mail.	5
HU.13	Como usuario, quiero que la pantalla de consulta esté centrada en el formulario activo.	2

HU.14	Como usuario quiero poder ver las respuestas dadas en los anteriores formularios para mejorar la calidad de mis respuestas.	4
HU.15	Como usuario, quiero poder cerrar mi sesión de forma segura para que nadie más pueda acceder a los datos introducidos.	1
HU.16	Como usuario, quiero tener la opción de poder NO responder a preguntas realizadas en los formularios	2
HU.17	Como usuario, quiero que el sistema me informe de los posibles errores	4

Cód H.U.	Criterios de aceptación
HU.1	☐ Debe haber un formulario estático de 5 campos obligatorios (departamento, patrimonio afectado, tiempo afectado). ☐ Deben haber la suficiente cantidad de formularios para que el usuario describa el problema a la aplicación (modelo IA).
HU.2	☐ La interfaz debe de ser capaz de construir automáticamente los formularios en base al JSON que devuelve el backend. ☐ Se debe poder mostrar distintos tipos de entrada de datos según la pregunta de la IA (campos texto, selección única, etc.) ☐ Los campos de los formularios deben incluir validaciones básicas si lo requieren.
HU.3	☐ El sistema debe guardar las respuestas del usuario en la base de datos cada vez que se envía un formulario o se avanza al siguiente paso. ☐ La sesión debe de estar marcada en un estado de progreso hasta que se genere el informe final para así diferenciarlas de las sesiones ya finalizadas. ☐ Al retomar la sesión, la aplicación debe cargar la sesión que quedó pendiente por finalizar sin que tenga que comenzar desde el principio el usuario.
HU.4	☐ La IA debe consultar cuando se envía el formulario, si la información que se tiene ya es suficiente para generar una propuesta. ☐ El sistema debe de preguntar un rango entre 10 y 30 preguntas respondidas en total.

	☐ El sistema debe de preguntar un rango entre 2 y 5 preguntas por cada formulario. ☐ Una vez se marca la completitud de información, no se debe generar nuevos informes y se debe indicar que se está generando la propuesta de solución. ☐ La sesión debe marcarse internamente en la base de datos como Finalizada, impidiendo que se puedan añadir más respuestas o modificar el formulario.
HU.5	☐ La propuesta solo se debe generar cuando la ingesta de datos ha finalizado. ☐ El texto generado de la propuesta debe de tener en cuenta toda la información que se ha recopilado durante la sesión. ☐ La propuesta generada debe guardarse en la base de datos y quedar asociada a la sesión actual. ☐ El usuario debe de poder leer la propuesta desde la interfaz de la aplicación una vez generada.
HU.6	☐ Debe haber un área de texto para introducir el nombre completo. ☐ Debe haber un área de texto para introducir el email. ☐ Debe haber un área de texto para introducir la contraseña ☐ El botón de enviar debe de estar desactivado hasta que no: ☐ El campo de email tenga un formato válido (ej: usuario@dominio.com). ☐ La contraseña cumpla con la longitud mínima de seguridad (ej: 8 caracteres) y sea alfanumérica. ☐ Todos los campos estén rellenos. ☐ Al darle al botón de enviar, debe aparecer un indicador de carga para informar al usuario que se está procesando la información y bloquearse los campos para evitar envíos dobles mientras el sistema procesa la solicitud.

	☐ Si el sistema detecta que el correo electrónico ya existe en la base de datos, debe mostrar un mensaje de error específico ("Este usuario ya está registrado") y permitir al usuario corregir el email sin borrar el resto de datos. ☐ Si el servicio de autenticación falla (caída de servidor o timeout), debe mostrar un mensaje de error amigable ("No se pudo crear la cuenta, inténtalo más tarde") manteniendo los datos del formulario visibles. ☐ Una vez creada la cuenta correctamente, el sistema debe mostrar una confirmación de éxito y redirigir automáticamente al home mostrando una. ☐ Se debe preguntar inicialmente información básica de la empresa que será guardada (como nombre empresa, sector en el que trabaja, nº personas empresa, ubicación geográfica, sociedad, B2B/B2C , antigüedad)
HU.7	☐ El sistema debe redirigir automáticamente a la pantalla principal (Dashboard/Home). ☐ El menú de navegación debe actualizarse para mostrar opciones privadas (ej: "Cerrar Sesión", "Mis Consultas"). ☐ Ante credenciales inválidas: ☐ El sistema debe mostrar un mensaje de error visual claro (ej: <i>"Usuario o contraseña incorrectos"</i>). ☐ El usuario debe permanecer en la pantalla de login para volver a intentarlo. ☐ El botón de "Entrar" debe estar deshabilitado si los campos de correo o contraseña están vacíos. ☐ El Backend debe responder con un código HTTP 200 y un JSON Web Token (JWT) en el cuerpo de la respuesta. ☐ El JWT debe tener una fecha de expiración configurada. ☐ El Frontend (Angular) debe capturar el token y almacenarlo. ☐ Si el usuario recarga la página (F5), la sesión debe mantenerse activa leyendo dicho token. ☐ Si un usuario ya logueado intenta entrar de nuevo a /login, el sistema debe redirigirle a /dashboard.

	☐ Si el token caduca el usuario debe ser redirigido de vuelta a /login.
HU.8	☐ Debe existir una sección llamada "Mis Informes" o "Historial" accesible desde el menú principal. ☐ Esta vista debe mostrar una lista de las consultorías finalizadas. ☐ Si el usuario no tiene informes, debe mostrarse un mensaje amigable (ej: <i>"Aún no has realizado ninguna consultoría"</i>). ☐ Se debe mostrar el texto completo de la Propuesta de Solución generada por la IA. ☐ Para mostrar el listado: ☐ El Backend debe proveer un endpoint que devuelva solo los informes pertenecientes al usuario autenticado. ☐ La respuesta debe ser un Array de objetos ligeros (sin todo el contenido del chat para no sobrecargar la lista). ☐ Para el informe detallado: ☐ El Backend debe proveer un endpoint para traer el contenido completo de un informe específico.
HU.9	☐ En la propuesta generada debe aparecer una explicación breve de por qué se ha generado esa propuesta. ☐ Si la IA finalmente no pudo encontrar una solución, debe indicarlo honestamente en lugar de inventar una justificación falsa.
HU.10	☐ En cada nueva llamada a la IA, el backend debe incluir obligatoriamente el historial completo de los pasos anteriores de esa sesión. ☐ La IA debe ser capaz de identificar que un dato ya ha sido proporcionado aunque la redacción sea distinta, de igual manera no puede preguntar lo mismo pero con redacción distinta. ☐ La memoria debe persistir si el usuario cierra el navegador y vuelve más tarde.
HU.11	☐ El informe debe tener el problema inicial y la propuesta (con la justificación).

	☐ Debe mostrarse con el markdown correspondiente para que sea fácil la lectura. ☐ Al pulsar el botón de generar informe, debe mostrarse un estado de "Generando documento..." bloqueando el botón para evitar múltiples clics.
HU.12	☐ Debe de haber un botón para descargar el informe con formato pdf. ☐ El archivo descargado/enviado debe tener un nombre significativo y único por defecto ☐ Debe haber un botón para enviarlo por mail.
HU.13	☐ Al cargar un nuevo formulario, la vista debe hacer scroll automático hacia abajo para enfocar el nuevo formulario. ☐ El formulario actual debe aparecer al final de la lista (parte inferior) y estar claramente resaltado o activo. ☐ Los controles de respuesta (campos de texto, botones de opción, etc.) deben ser los únicos elementos interactivos presentes, además de los botones de acción necesarios (ej: "Enviar", "Siguiente").
HU.14	☐ La pantalla debe presentar los elementos en una lista vertical. ☐ Las interacciones con los formularios pasados deben aparecer por encima del formulario actual. ☐ El usuario debe poder hacer scroll manual hacia arriba para leer todo el historial anterior sin límites. ☐ Los inputs de las preguntas pasadas deben mostrarse en estado "de solo lectura". ☐ El historial debe respetar el tipo de respuesta que se dio. Si la respuesta anterior fue un "Select" (opción múltiple), en el historial se debe ver la opción elegida claramente.
HU.15	☐ Debe existir un botón o opción de "Cerrar Sesión" claramente visible y accesible desde la barra de navegación en todas las pantallas de la zona privada. ☐ Al pulsar el botón, la aplicación debe eliminar inmediatamente el token de autenticación (JWT) y cualquier información sensible almacenada en el navegador.

	☐ Una vez finalizado el proceso de cierre, el sistema debe redirigir al usuario automáticamente a la pantalla de Login. ☐ Si un usuario intenta volver a la pantalla anterior pulsando el botón "Atrás" del navegador después de cerrar sesión, el sistema debe interceptar la navegación y redirigir de nuevo al Login.
HU.16	☐ Debe existir una opción clara y visible, como un botón o casilla etiquetada como "Omitir", para algunas preguntas que el usuario pueda saltar. ☐ La opción de "Omitir" no debe estar disponible para aquellas preguntas que sean marcadas como obligatorias por la lógica de negocio o la IA. ☐ El botón de enviar debe estar desactivado hasta que no se completen las preguntas obligatorias. ☐ Si una pregunta es opcional, la opción de "Omitir" debe permitir el avance al siguiente paso o formulario. ☐ Al omitir una pregunta, el sistema debe almacenar explícitamente el valor como "omitido" en la base de datos para diferenciarlo de una respuesta. ☐ La lógica del Agente de IA debe ser capaz de manejar la ausencia de la respuesta (el valor "omitido").
HU.17	☐ En caso de error mostrar retroalimentación al usuario de que el sistema está fallando.

TAREAS

Esta sección traduce los requisitos funcionales y los Criterios de Aceptación en un plan de acción técnico y priorizado. Estas tareas forman la base para la planificación de los sprints de desarrollo y ayudan a coordinar los esfuerzos entre los equipos de frontend, backend e integración. Para la fase inicial, se priorizará un 'slice' vertical que abarque el flujo completo de autenticación (Épica 1) y un ciclo simple de consulta-respuesta con el motor de formularios dinámicos (Épica 2), permitiendo una validación temprana del core de la aplicación.

Tareas de configuración y seguridad

Backend	☐ Configuración inicial del servidor Express y API REST. Descripción: Estructurar el proyecto, configurar middleware esencial (CORS, body-parser) y definir el enrutador principal de la API. ☐ Diseño del esquema de base de datos. Descripción: Crear el modelo E/R de la BD ☐ Configuración de la conexión a la BD con ORM. ☐ Implementación del esquema de base de datos. Descripción: Crear los modelos de datos para Usuarios, Sesiones y Respuestas De Formulario utilizando un ORM o cliente de base de datos. ☐ Configurar .env-template con los datos necesarios
Frontend	☐ Configuración inicial del proyecto Angular. Descripción: Estructurar el proyecto con módulos, enrutamiento, servicios y configuración de entorno. ☐ Configuración de la conexión con el backend (desde el back). ☐ Implementación de Route Guards para proteger rutas. Descripción: Crear guards de Angular para prevenir el acceso a rutas privadas si el usuario no está autenticado y redirigir si el token ha caducado.
Agente	
Test	☐ Pruebas de Conexión: Unit Test: Verificar que la función de conex ☐ ión devuelve un estado "Connected". ☐ Integration Test: Realizar un INSERT de prueba en SQL, y borrarlos inmediatamente (Cleanup). ☐ Validación de Esquema: Comprobar que los campos obligatorios (como el email en SQL o el sessionId en Mongo) arrojan error si se envían nulos.

HU.1

Backend	☐ Endpoint inicio auditoría (<i>POST /api/session/start</i>). Desarrollar lógica para guardar la nueva sesión creada así como el manejo de todos los JSON que se le pasará al agente.
Frontend	☐ Crear componentes para mostrar los formularios dinámicos y manejar el envío al backend de estos. ☐ Se debe validar que las preguntas obligatorias hayan sido respondidas correctamente.
Agente	☐ Instruir a la IA que en función de los campos rellenos, decida sobre qué banco de preguntas va a realizar las siguientes preguntas para tener información más precisa.
Criterios Calidad	☐ Mostrar pantalla de inicio de la auditoría con las primeras preguntas estáticas listas para responder. ☐ Indicación de espera al enviar el formulario. ☐ Si le da a enviar sin rellenar los campos obligatorios, muestra que debe de rellenarse obligatoriamente. ☐ El formato de las respuestas debe ser validado
Test	

HU.2

Backend	☐ Definir Contrato JSON (Schema): Crear la estructura estándar de respuesta (ej. input_type, label, validation_rules, options) que el Frontend consumirá. ☐ Endpoint de Orquestación: Implementar endpoint (ej. POST /conversation/next) que reciba el historial(con el estado de la sesión), llame al Agente y devuelva el JSON del formulario. ☐ Validación de Esquema: Asegurar que la salida del Agente cumpla estrictamente con el contrato UI antes de enviarla al front (evitar errores de renderizado).
Frontend	☐ Manejo de Estado y Validaciones: Implementar lógica (ej. usando React Hook Form o Zod) para leer las reglas del JSON

	(ej. required: true) y bloquear el envío si no se cumplen. ☐ Feedback Visual: Implementar estados de carga (skeletons) mientras el agente "piensa" y genera el siguiente formulario.
Agente	☐ Prompt Engineering para UI: Diseñar el prompt del sistema para que no responda con texto plano, sino que decida qué dato falta y elija el widget de UI adecuado para pedirlo (ej. "Si necesitas una fecha, usa date-picker, no text"). ☐ Structured Output: Configurar el modelo (usando Function Calling o librerías como Instructor) para garantizar que la salida sea un JSON válido parseable por el Backend.
Criterio Calidad	Integridad del JSON: El formulario se renderiza sin errores de "undefined" o llaves rotas. Adaptabilidad: Si el agente pide una lista de opciones, el frontend muestra un Dropdown; si pide una descripción, muestra un Textarea. Robustez: El usuario no puede avanzar a la siguiente pregunta sin satisfacer las validaciones del formulario actual.
Test	

HU.3

Backend	☐ Endpoint para guardar y continuar sesiones (PUT /api/consultations/:id/sync). Descripción: Desarrollar la lógica para recibir el historial actual o el último mensaje del usuario y actualizar el registro en la base de datos. ☐ Endpoint de Recuperación de Sesión Activa (GET /api/consultations/active). Descripción: Crear una consulta que busque en la tabla consultations si existe algún registro asociado al usuario con status: 'IN_PROGRESS'. Si existe, debe devolver el historial completo (history) y el id de la sesión.
Frontend	☐ Implementar Servicio de Persistencia Local (Frontend Service). Descripción: Desarrollar servicio en Angular (StorageService) que abstraiga el uso de localStorage. Debe incluir métodos para guardar borradores (saveDraft), obtenerlos (getDraft) y limpiarlos. ☐ Implementar Orquestación de Carga de Sesión (Frontend - ngOnInit). Descripción: Desarrollar la lógica de inicio que decida qué

	datos mostrar: primero verifica si hay un borrador local, si no, llama al endpoint GET /active. Si no hay nada, inicia una sesión limpia.
Agente	
Criterio Calidad	☐ Los datos se guardan en el navegador y persisten aunque se cierre la pestaña. ☐ Al recargar la página (F5), el formulario muestra exactamente lo que el usuario estaba escribiendo antes de salir. ☐ El almacenamiento local queda vacío para evitar que aparezcan datos antiguos en la siguiente pregunta.
Test	

HU.4

Backend	☐ Implementación lógica conteo y cierre de sesión. Descripción: Desarrollar el servicio que valide los límites de $10 \leq \text{nº preguntas} \leq 30$, gestione el flag de finalización recibido de la IA y actualice el estado de la sesión a "Finalizada" en base de datos.
Frontend	☐ Gestión de estado "completado". Descripción: Detectar cuando se indica fin de realización de preguntas, para generar la propuesta final bloqueando la escritura en los formularios.
Agente	☐ Prompt de "El Crítico": Configurar un paso de razonamiento previo donde se le inyectan los contadores. Regla: "Si total_questions < 10, DEBES seguir preguntando. Si total_questions > 25, intenta cerrar ya. Si total_questions == 30, CIERRA OBLIGATORIAMENTE." ☐ Control de Output: Instruir al Agente para que, al generar un formulario, genere estrictamente entre 2 y 5 campos.
Criterio Calidad	Límite Inferior: El sistema NUNCA propone una solución con menos de 10 respuestas (aunque crea saber la solución, debe profundizar para cumplir el criterio). Límite Superior: El sistema NUNCA supera las 30 preguntas. Si llega a 30, debe ofrecer la mejor solución parcial posible. Integridad: Una vez finalizado, un intento de POST debe de rechazarlo completamente

Test	
------	--

HU.5

Backend	☐ Persistencia de Propuesta: Crear el campo proposal_content (Text/JSON) en la tabla de Sesiones y un endpoint GET /session/proposal para servirlo al front una vez listo y POST para guardarlo. Pasar id como query a las rutas ☐ Servicio de Generación de Informes (Prompting Avanzado). Descripción: Crear un método específico en AiService (ej: generateFinalReport(history)). ☐ Orquestación del Cierre (Trigger). Descripción: Conectar la lógica de "Fin de sesión" (definida en la HU anterior) con la generación del informe. ☐ Agregación de contexto y generación de la propuesta : Descripción: Lógica de recopilación de todo el historial de la sesión, envío al modelo de IA para generar la propuesta así como el guardado de esta en base de datos.
Frontend	☐ Componente con la visualización de la propuesta final. Descripción: Renderizar informe recibido formateando el texto en MD (Librería Marked).
Agente	Lógica del Prompt: Debe inyectar todo el history como contexto. System Prompt aprox: "Actúa como un consultor senior. Basado en la siguiente entrevista, genera un informe profesional estructurado en formato Markdown. Incluye: 1. Resumen Ejecutivo, 2. Diagnóstico del Problema, 3. Solución Propuesta con justificación del porqué
Criterio Calidad	Integridad de Datos: La propuesta no debe dejar fuera restricciones críticas (ej. si el usuario dijo "Plazo: 1 semana", la solución no puede ser un plan a 6 meses). Latencia Percibida: El usuario debe tener feedback visual constante durante la generación (ej. "Analizando respuestas...", "Redactando estrategia..."). Persistencia: La propuesta generada es inmutable. Si recargo

	la página, veo exactamente el mismo texto, no se regenera de nuevo (ahorro de costes). La visualización debe respetar el formato de texto parseado. El contenido generado de la propuesta tiene que estar relacionado con los datos suministrados durante la sesión
Test	

HU.6

Backend	☐ Endpoint de registro de usuario (POST /api/auth/register). Descripción: Desarrollar la lógica para crear un nuevo usuario, incluyendo hash de contraseña y validación de email único.
Frontend	☐ Crear el componente de registro. Descripción: Desarrollar el formulario con validaciones en el lado del cliente.
Agente	
Criterio Calidad	☐ El diseño debe ser responsive y los campos deben tener etiquetas (labels) claras. ☐ Uso de tipos de input correctos (email, password). ☐ El botón de enviar debe permanecer desactivado hasta que el formulario sea 100% válido. ☐ Los mensajes de error de validación deben ser visibles al interactuar con los campos. ☐ En caso de error, el formulario no debe resetearse (excepto contraseña). ☐ El indicador de carga debe desaparecer una vez finalizada la respuesta (éxito o error) ☐ La redirección debe ser automática tras el mensaje de éxito.
Test	

HU.7

Backend	☐ Endpoint de autenticación y generación de JWT (POST /api/auth/login). Descripción: Implementar la lógica de login que verifica
----------------	--

	credenciales y genera un JSON Web Token con una fecha de expiración.
Frontend	☐ Crear el componente formulario del login, con validaciones del lado del cliente.
Agente	
Criterio Calidad	☐ El JWT debe estar firmado con una clave secreta almacenada en variables de entorno (.env) ☐ No se deben devolver datos sensibles (como la contraseña) en el JSON de respuesta
Test	

HU.8

Backend	☐ Endpoint para listar informes finalizados (GET /api/reports). Descripción: Crear un endpoint protegido que devuelva una lista de los informes finalizados para el usuario autenticado. ☐ Endpoint para obtener el detalle de un informe (GET /api/reports/:id). Descripción: Proveer un endpoint que devuelva el contenido completo de un informe específico, incluyendo el historial de la consulta y la propuesta final.
Frontend	☐ Implementar un dashboard a modo de tablero que lista las consultorías ya realizadas. ☐ Implementar un modal que despierta un dialog con el informe de la propuesta detallado. En ese mismo dialog los botones de descargar PDF o enviar por mail.
Agente	
Criterio Calidad	☐ Si no hay informes todavía no debe mostrarse del dashboard ☐ Al entrar en la URL /dashboard/report/123, se carga y visualiza el informe formateado correctamente. ☐ La navegación entre la lista y el detalle es fluida y la URL cambia correctamente. ☐ El usuario ve sus informes listados o el mensaje de bienvenida si es nuevo.
Test	

HU.9

Backend	
Frontend	
Agente	☐ Prompt de Justificación: Instruir al modelo para que genere en la propuesta la justificación que vincule explícitamente a la descripción del problema del usuario. ☐ Si considera que no ha tenido suficientes datos pero ha llegado a su cupo de preguntas debe justificar correctamente porque no ha podido generar una propuesta.
Criterio Calidad	☐ Trazabilidad: La explicación debe hacer referencia al menos a un dato proporcionado por el usuario (no valen justificaciones genéricas como "Es la mejor opción"). ☐ Claridad en el Fallo: Si no hay solución, el mensaje debe ser constructivo, explicando qué restricción bloqueó el proceso.
Test	

HU.10

Backend	☐ Persistencia de Sesión (DB): Diseñar esquema en base de datos para almacenar el historial de mensajes (chat_history) y el estado actual de las variables (extracted_data) vinculado a un session_id. ☐ Endpoint de Recuperación: Crear endpoint GET /session con las preguntas y respuestas realizadas en los anteriores formularios. Pasar id como query a las rutas
Frontend	☐ Gestión de Session ID: Implementar lógica en localStorage para guardar el identificador de la sesión actual y recuperarlo al recargar la página. ☐ Renderizado de Historial: Al iniciar la app, si existe una sesión

	previa, cargar y pintar los mensajes anteriores para dar continuidad visual.
Agente	☐ System Prompt: "Analiza el historial para no volver a preguntar preguntas ya respondidas y que no haya deduplicación semántica."
Criterios Calidad	☐ El sistema no repite la misma pregunta con otras palabras.
Test	

HU.11 **EXPLICAR PEDRO ISAAC ambas cosas**

Backend	☐ 1. Endpoint de Reporte: Crear ruta GET /session/report que recupere el contexto completo (problema inicial + propuesta) y genere el informe en formato .pdf. ☐ Endpoint para resumir la sesión y que devuelva el problema que tiene la empresa en un párrafo GET /session ☐ Pasar id como query a las rutas
Frontend	☐ Mostrar al usuario que se está generando el informe. ☐ Integrar MD como estilos para el informe generado. ☐ Renderizar una vista previa del informe.
Agente	☐ Resume la sesión en un párrafo explicando el problema de la empresa
Criterio Calidad	☐ Bloqueo Efectivo: Es imposible disparar dos peticiones seguidas (doble clic) ☐ Legibilidad: El reporte no es un bloque de texto plano; usa correctamente los estilos de MD. ☐ Completitud: El reporte incluye el problema original de la empresa(no solo la solución).
Test	

HU.12

Backend	☐ Generador de Nombres Únicos: Implementar lógica para definir el nombre del archivo:: Reporte_{Nombre_Empresa}{Fecha_Creacion}.pdf. ☐ Servicio de Email: Integrar proveedor transaccional y crear el endpoint POST /report/email que adjunte el PDF generado. ☐ Asunto del Email: Generar dinámicamente el Subject y el Body del correo para que sea personalizado.
Frontend	☐ UI de Envío: Implementar un modal al pulsar "Enviar por mail" para confirmar la dirección de destino (prellenada si el usuario está logueado). ☐ Feedback de Proceso: Gestionar los estados de espera distintos: "Preparando PDF..." para descarga (en caso de que el usuario quiera) y "Enviando correo..." para el mail.
Agente	
Criterio Calidad	☐ Fidelidad Visual: El PDF debe mantener el formato del Markdown (negritas, listas, saltos de línea) y no verse como texto plano roto. ☐ Seguridad: Validar formato de email en backend antes de intentar el envío. ☐ Debe haber un botón claro de envío por correo del pdf generado
Test	

HU.13

Backend	☐ Indicar con una “flag” (LAST_QUESTION) cuál es el último formulario que ha generado el agente.
Frontend	☐ Gestión de Foco Automático (Auto-Scroll) ☐ Que se vea visualmente diferente el que tenga flag de última pregunta
Agente	
Criterio Calidad	Foco del Cursor: Al cargar el nuevo formulario, el cursor del teclado debe aparecer automáticamente en el primer input (autofocus), ahorrando un clic al usuario. Visibilidad: El nuevo formulario debe de ocupar el máximo espacio posible.
Test	

HU.14

Backend	☐ Para cuando se pierde la sesión llamar a un endpoint /api/sessions/history. Pasar id como query a las rutas Descripción: devuelve todas las preguntas con sus respuestas hechas anteriormente para rehacer la sesión.
Frontend	☐ Para cuando la sesión activa ir guardando las preguntas y respuestas en un array para poder ver preguntas y respuestas anteriores. ☐ ChatHistoryComponent:Carga la vista del formulario y preguntas del array o del endpoint ☐ Las preguntas anteriores estarán en modo lectura
Agente	
Criterio Calidad	Bloqueo Efectivo: No debe ser posible modificar una respuesta de los formularios que ya han sido enviados anteriormente. No volver a cargar los formularios ya renderizados en el frontend anteriormente.
Test	

HU.15

Backend	☐ Middleware de protección de rutas con JWT. Descripción: Crear un middleware reutilizable para verificar la validez del JWT en las rutas que requieran autenticación. ☐ Cierre de Streams: Si hay una generación de IA en curso asociada a ese usuario, enviar una señal de cancelación para detener el proceso y no consumir recursos inútilmente.
Frontend	☐ Purga de Almacenamiento Local: Crear una función utilitaria clearSession() que elimine localStorage. ☐ Redirección Forzada: Enviar a la página de inicio de sesión de nuevo. ☐ Debe haber un boton visible siempre de logut
Agente	
Criterio Calidad	Prueba del "Botón Atrás": Una vez hecho el logout, si el usuario pulsa el botón "Atrás" del navegador, NO debe ver la pantalla anterior con datos, sino ser redirigido al login. Inutilidad del Token: Un token capturado antes del logout debe dejar de funcionar inmediatamente después de llamar al endpoint de salida. 3. Feedback: El usuario recibe confirmación visual de que ha salido ("Has cerrado sesión correctamente").
Test	

HU.16

Backend	
Frontend	☐ Las que se puedan omitir no estarán marcadas con * ☐ Botón de omitir la pregunta realizada
Agente	☐ Prompt de Tolerancia: Instruir al modelo en el System Prompt: "Si la pregunta ha sido omitida NO vuelvas a preguntar lo mismo. Asume que el dato no estará disponible y busca

	soluciones generalistas que no dependan de esa variable". ☐ Generación de UI: Entrenar/Instruir al agente para que decida cuándo una pregunta es crítica (obligatoria) y cuándo es accesorio (opcional) al generar el JSON del formulario.
Criterio Calidad	Persistencia Explícita: En la base de datos, un campo omitido no es NULL (que podría significar "aún no preguntado"). Debe ser un valor explícito que indique "El usuario decidió no responder". UX de Bloqueo: Es visualmente imposible omitir un campo obligatorio (el botón no existe) y es imposible enviar el formulario si un campo obligatorio está vacío. Flujo Continuo: Pulsar "Omitir" se siente tan fluido como pulsar "Siguiente" (avanza inmediatamente en flujos paso a paso).
Test	

HU.17

Backend	Middleware global para manejo de errores: Descripción: Implementar middleware para capturar los errores (código de error y msj amigable).
Frontend	Interceptar errores http con su correspondiente notificación Descripción: Crear un interceptor de códigos de error y mostrarlo con un componente de errores junto al mensaje
Agente	
Criterios Calidad	☐ Los mensajes de errores deben ser claros y comprensibles de cara al usuario. ☐ Notificaciones temporales o con botón de cierre. ☐ Se debe evitar el bloqueo. ☐ Registrar logs en servidor??
Test	

Boceto del Frontend

La interfaz de usuario (UI) es un componente crítico para el éxito de este proyecto. Dado que el objetivo es una aplicación "agentic", la experiencia debe ser fluida, intuitiva y conversacional, guiando al usuario a través de un proceso de consulta potencialmente complejo sin generar fricción. Una UI limpia y modular es fundamental para lograrlo.

El diseño conceptual de la interfaz se centrará en una vista de lista vertical, similar a una aplicación de chat. Los formularios dinámicos, generados por el backend, aparecerán secuencialmente a medida que el usuario avanza en la consulta. El formulario activo estará siempre enfocado en la parte inferior de la pantalla, presentando los campos de entrada y los botones de acción como únicos elementos interactivos. Las preguntas y respuestas de los pasos anteriores serán visibles en la parte superior de la vista en un estado de solo lectura, permitiendo al usuario revisar el historial completo de la conversación en cualquier momento.

A continuación, se proporciona acceso al prototipo visual que plasma estas ideas: Acceso al prototipo en Figma: [Boceto FIGMA](#)

Para materializar este diseño, es fundamental desglosar las responsabilidades técnicas del frontend y backend en un plan de acción concreto.

Tecnologías a usar

El desarrollo se fundamentará en un stack tecnológico robusto compuesto por Node.js y Express para la arquitectura del backend, y Angular en el frontend para la gestión avanzada de formularios dinámicos. La seguridad y el control de acceso se garantizarán mediante autenticación basada en JWT.